



BSc (Hons) in **Information Technology**  
**Specialized in AI**

## **IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## **Practical 01**

**Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

### **Part 1: Code Analysis & PEAS Evaluation**

#### **Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

#### **Task 1.1: The Environment State (`__init__`)**

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance measure  
E – Environment  
A – Actuators  
S – Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

Opponent positions (self.opponents)

These variables describe the state of the external world in which the agent operates

**3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.**

The environment contains self.opponents, meaning other entities exist and can perform actions that influence the environment. an environment is multi-agent when other entities are present and pursuing their own behaviors.

### Task 1.2: The Perception Subsystem (get\_percept)

**4. (Remember) Define what a "percept sequence" is according to Lecture 01.**

A percept sequence is the complete historical record of all percepts received by the agent from initialization up to the current moment.

**5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?**

Sensors (S).  
`get_percept()` gathers information from the environment and provides it to the agent

**6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.**

states, or information not included in the percept dictionary), its sensors do not provide complete knowledge of the environment. Therefore, the environment is partially observable.

### Task 1.3: Action Execution (execute\_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P).

The score represents how well the agent is performing, so deducting points updates the performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

metric by maximizing internal activity rather than achieving the intended goal.

Lecture 02's Vacuum World example shows how a poor metric can lead to irrational behavior that scores well but fails the real objective

## Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

### Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

important information affecting agent performance that cannot be detected through its sensors. The agent must make decisions without full knowledge of relevant environmental states

### Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

about toxic locations. A rational agent chooses actions that maximize expected performance. Since traps cause penalties, detecting them allows the agent to avoid harmful actions and select safer alternatives, thereby increasing expected utility

### Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

This generated unlimited points without actually keeping the environment clean. The corrected metric rewarded maintaining a clean environment over time

Finalize and Lock Lab 01 Analytical Evaluation



**FACULTY OF COMPUTING**